IT 360 Cloud Integration | Week 3 | Team Lab 1 (Canvas 3.2, 40 points)

Lab 1: Kubernetes on GKE with Helm and a NetworkPolicy

Silver Lake Credit Union: run the loan calculator on Google Kubernetes Engine, roll it back with one command, and prove that only the front end can reach it

---

Version 2, November 2026. Every command in this guide was run end to end on November 8, 2026, and the "You should see" boxes show the real output of that run. Your pod names, IP addresses and times will differ; the shape of the output will not.

> **Duration: about 80 minutes in total (75 in class plus 5 to get ready)**
>
> **Before class:** 5 minutes (start the Vocareum lab, open Cloud Shell).
>
> **In class:** 75 minutes of lab work in six parts (A 15, B 10, C 15, D 15, E 10, F 10). Part A is mostly waiting for Google to build the cluster; read Part B while it runs.
>
> **Instructor's timed run (November 8, 2026):** all four build parts, cluster included, took under 8 minutes of machine time; the cluster was RUNNING in under 5 minutes, the replacement pod appeared in 2 to 8 seconds, and the optional public IP arrived in 44 seconds.
>
> **Allow extra time if:** Google reports no capacity in your region (recreating the cluster adds 10 to 25 minutes) or Autopilot is adding a machine for the test pods (up to 90 seconds each).

The story

Silver Lake Credit Union is moving its public loan calculator onto Google Kubernetes Engine (GKE). Last year an auditor wrote: "public-facing workloads must not be able to reach internal workloads on the same network." The infrastructure manager wants three things: the calculator running on Kubernetes, a release that can be rolled back with one command, and **proof** that pods without permission cannot reach it. A blocked connection is the proof.

What you will build

Part	What you do	Time
A. Cluster	Create a GKE Autopilot cluster named silverlake-public	15 min (mostly waiting)
B. kubectl	Run 3 copies of the web server and watch Kubernetes heal itself	10 min
C. Helm	Install the app as a Helm release, upgrade it, roll it back	15 min
D. NetworkPolicy	Allow only role=frontend pods in, and prove it with two tests	15 min
E. Connectivity	Test every network path, including from the internet (optional)	10 min
F. Submit and clean up	Screenshots to Canvas, then delete the cluster	10 min
Total	Six parts in one lab session	about 75 min, plus 5 min before class

Team roles

Role	Owns
Cluster engineer	Part A: creates the cluster and connects kubectl
Deployment engineer	Part B: applies the Deployment and runs the self-healing check
Release engineer	Part C: the Helm install, upgrade and rollback
Security engineer	Parts D and E: the policy, the two probes, the screenshots for the auditor

Everyone types along in their own Cloud Shell if they can, but the role owner drives that part on the shared screen.

Before you start (5 minutes)

- ☐ Open Canvas and start the **Vocareum Google Cloud lab**, then click through to the Google Cloud console. (Using your own Google Cloud project instead? That works too; you need billing turned on.)
- ☐ At the top of the console, check the project selector shows your lab project.
- ☐ Open **Cloud Shell**: the >_ icon at the top right of the console. A terminal opens at the bottom of the page. Everything you type in this lab goes there.
- ☐ Cloud Shell already has gcloud, kubectl and helm. You install nothing, and it works the same on a Mac or Windows laptop because it runs in your browser.
- ☐ Bookmark the class repository: **https://github.com/isijackson/IT360-Fall-Labs-v2**. The lab files are in Week 3 labs/gke-helm-networkpolicy (the guide downloads the two YAML files from there for you in Parts B and D).
- ☐ Fill in the table below with your values.

Item	Value in this guide	Yours
Project ID	jackson-isiko	
Region	us-central1 (if it is full, use us-east1)	
Cluster	silverlake-public	keep this name
App names	loan-calc (do not change)	keep these names

Where you run things

Place	How to open it	What you do there
Cloud Shell (a Linux terminal inside your browser)	Google Cloud console, the >_ icon at the top right. It opens in your **home folder**; the prompt then ends in cloudshell:~ (your-project)$. If it shows cloudshell:/ or any other folder, type cd ~ to return home.	**Every command in this guide**: gcloud, kubectl, helm, curl, sed. Paste one block at a time and press Enter.
Google Cloud console (web pages)	https://console.cloud.google.com, the same browser tab	The **Click** steps: Kubernetes Engine, Clusters, Workloads, Services

Place	How to open it	What you do there
Your laptop's web browser	Any tab	Part E only: open the public EXTERNAL-IP address
Your laptop's own Terminal or PowerShell	Not needed	Nothing. Do not run these commands on your laptop; Cloud Shell already has every tool and is signed in to your project.

Every command block below starts with an orange ▸ **Run in:** line that tells you exactly where to run it.

> **How to read this guide**
>
> **Click:** numbered console steps, with a screenshot where it helps. The red numbers in the screenshot match the step numbers.
>
> **Type:** the same thing as commands for Cloud Shell. Copy one block at a time and press Enter.
>
> **You should see:** green boxes with the real output from the November 8 run.
>
> **If not:** yellow boxes with the fix for the problems students hit most often.
>
> Replace anything in angle brackets, like `<PROJECT_ID>`, including the brackets.

Part A. Create the cluster (Cluster engineer, 15 min)

Goal: a running Kubernetes cluster, and kubectl connected to it. Autopilot means Google manages the machines for you; it also enforces NetworkPolicy by default, which Part D needs.

Option 1: Click (the console)

1. Click the menu (three lines, top left), then **Kubernetes Engine**, then **Clusters**.
2. Click **Create**. If Google asks you to enable the Kubernetes Engine API, click **Enable** and wait about a minute.
3. Under **Autopilot**, click **Configure**. Do not pick Standard: the policy test in Part D can silently fail on Standard.
4. Name: silverlake-public. Region: us-central1. If a "Navigate at a glance" tour card covers the Name box, close it with its X.
5. Leave everything else as it is and click **Create** at the bottom.
6. Wait until the cluster shows a green check (5 to 15 minutes). Read Part B while you wait.

Verify (console)

Kubernetes Engine > Clusters lists **silverlake-public** with a **green check**, Mode **Autopilot**, Location **us-central1**. A spinner means it is still being created; keep waiting.

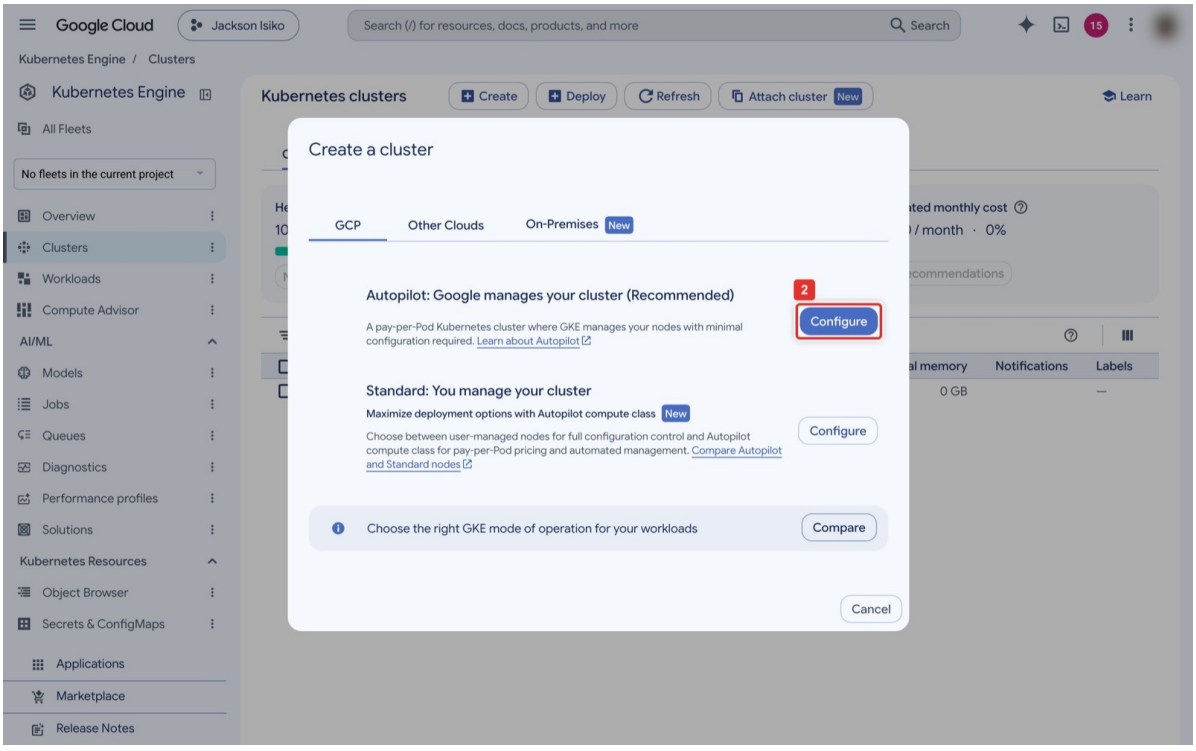


Figure 1. Step 3: click Configure under Autopilot.

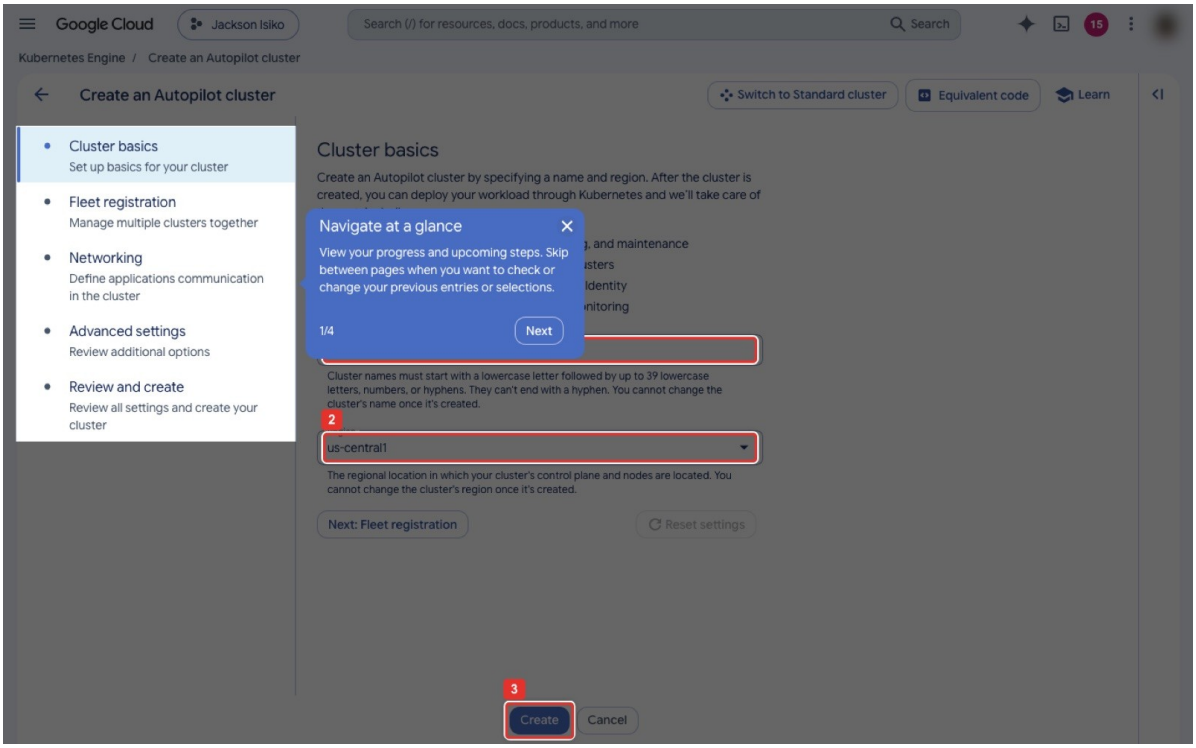


Figure 2. Steps 4 and 5: name, region, then Create at the bottom.

Option 2: Type (Cloud Shell), same result

▸ **Run in: Cloud Shell (Google Cloud console, the >_ icon at the top right). Any folder.**

```
gcloud container clusters create-auto silverlake-public --region us-central1
```

Then, either way: connect kubectl to the cluster

▸ **Run in: Cloud Shell (Google Cloud console, the >_ icon at the top right). Any folder.**

```
gcloud container clusters get-credentials silverlake-public --region us-central1
gcloud container clusters list
```

You should see

```
kubeconfig entry generated for silverlake-public.
NAME              LOCATION     MASTER_VERSION      STATUS
silverlake-public  us-central1  1.36.4-gke.2377000  RUNNING
```

STATUS **RUNNING** is Validation 1. Take a screenshot of the cluster list (Screenshot 1, part one).

If not

does not have enough resources available: Google has no room in that region right now. Delete the failed cluster (`gcloud container clusters delete silverlake-public --region us-central1 --quiet`) and create it again with `--region us-east1`. Use `us-east1` in every later command too.

INTERNAL error after 20+ minutes: a temporary Google problem. Delete the cluster and create it again.

kubectl prints HTML or "connection refused": you skipped get-credentials. Run it again.

Part B. Deploy with kubectl (Deployment engineer, 10 min)

Goal: three copies (pods) of the web server running, and proof that Kubernetes replaces a pod that dies.

Step 1. Get the Deployment file

Download it from the class repository on GitHub (one line, nothing to retype), then look at it:

▸ Run in: Cloud Shell (Google Cloud console, the >_ icon at the top right), in your home folder (where Cloud Shell opens; type cd ~ first if you moved). Paste the four lines one at a time; the two R= lines only build the address so it fits on the page. The file is saved in your home folder; ls shows it.

```
cd ~
R=https://raw.githubusercontent.com/isijackson/IT360-Fall-Labs-v2/main
R="$R/Week%203%20labs/gke-helm-networkpolicy/k8s"
curl -fsSO "$R/nginx-deployment.yaml"
cat nginx-deployment.yaml
```

Verify the download (Cloud Shell, home folder)

```
ls -l nginx-deployment.yaml
```

You should see a size of **344** bytes:

```
-rw-r--r-- 1 <you> <you> 344 Nov 10 09:00 nginx-deployment.yaml
```

A size of 0, or No such file or directory, means the download failed: run the curl line again.

Inside the file (for reading; YAML needs these exact spaces, so download it rather than copying it from a PDF):

Inside the file (for reading only; do not paste this into Cloud Shell)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: loan-calc
  labels:
    app: loan-calc
spec:
  replicas: 3
  selector:
    matchLabels:
      app: loan-calc
  template:
    metadata:
      labels:
        app: loan-calc
    spec:
      containers:
      - name: nginx
        image: nginx:1.27
        ports:
        - containerPort: 80
```

replicas: 3 is the important line: you are telling Kubernetes "always keep three of these running".

Step 2. Apply it and wait

▸ Run in: Cloud Shell (Google Cloud console, the >_ icon at the top right), in the same folder as nginx-deployment.yaml (your home folder).

```
kubectl apply -f nginx-deployment.yaml
kubectl rollout status deployment/loan-calc
kubectl get pods
```

You should see

```
deployment.apps/loan-calc created
deployment "loan-calc" successfully rolled out
NAME                        READY   STATUS    RESTARTS   AGE
loan-calc-78dcf96fdc-7ntzj   1/1     Running   0          2m58s
loan-calc-78dcf96fdc-89fvv   1/1     Running   0          2m58s
loan-calc-78dcf96fdc-9pnnh   1/1     Running   0          2m58s
```

A warning that starts autopilot-default-resources-mutator is normal: Autopilot filled in a CPU size for you. The first rollout can take 2 to 3 minutes while Autopilot adds a machine.

If not

curl prints `(23) Failed writing received data` or `(56) Failure writing output`, or `cat` says No such file: you are not in your home folder (the prompt shows cloudshell:/ instead of cloudshell:~), or the address has a typo such as a space in https. Run cd ~ and paste the lines again exactly as printed. Still failing? Create the same Deployment with one command: kubectl create deployment loan-calc --image=nginx:1.27 --replicas=3 (it uses the same name and the same app=loan-calc label).

error converting YAML / mapping values are not allowed: the file lost its spaces (usually copied from a PDF). Delete it (rm nginx-deployment.yaml) and download it with curl.

Step 3. Break it on purpose: delete a pod

Copy one pod name from your own list (not from this guide), then:

▸ **Run in: Cloud Shell (Google Cloud console, the >_ icon at the top right). Any folder.**

```
kubectl delete pod <one-pod-name>
kubectl get pods
```

You should see

```
pod "loan-calc-78dcf96fdc-7ntzj" deleted
loan-calc-78dcf96fdc-89fvv   1/1     Running   0          3m7s
loan-calc-78dcf96fdc-9pnnh   1/1     Running   0          3m7s
loan-calc-78dcf96fdc-knb66   1/1     Running   0          8s
```

Still three pods. The new one is only seconds old. Take **Screenshot 1** (three pods Running, one of them new).

Checkpoint: who restored the pod?

The **ReplicaSet** behind the Deployment. It saw two pods where you asked for three and started a new one. No person was involved. Be ready to explain this to the instructor.

Click view: Kubernetes Engine, then Workloads, then loan-calc shows the same three pods. You can delete a pod there too: click its name, then Delete.

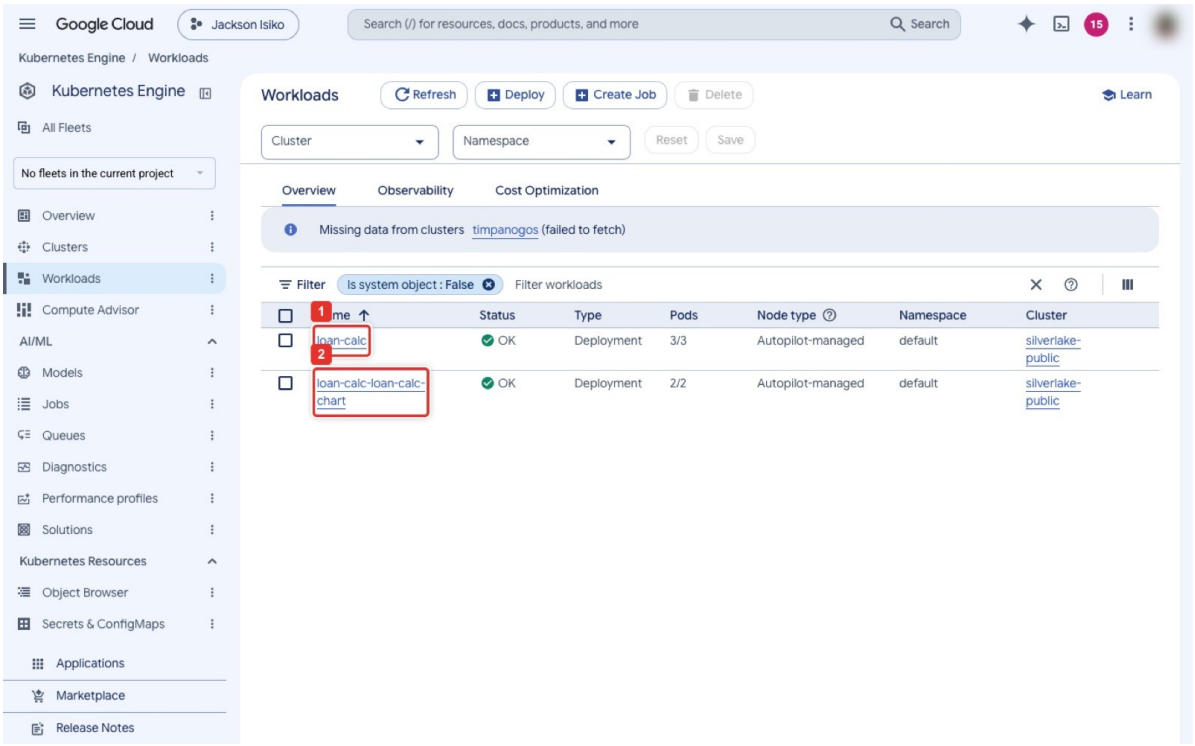


Figure 3. Kubernetes Engine > Workloads: loan-calc (Part B) and loan-calc-loan-calc-chart (Part C).

Part C. Manage the app with Helm (Release engineer, 15 min)

Goal: the same app installed as a Helm **release**, so that every change gets a revision number and any revision can be restored with one command.

Step 1. Create a chart and set two values

▸ Run in: Cloud Shell (Google Cloud console, the >_ icon at the top right), in your home folder (where Cloud Shell opens; type cd ~ first if you moved). helm create makes the loan-calc-chart folder there.

```
helm version --short
helm create loan-calc-chart
sed -i.bak 's/^replicaCount: .*/replicaCount: 2/' loan-calc-chart/values.yaml
sed -i.bak 's/^  tag: .*/  tag: "1.27"/' loan-calc-chart/values.yaml
```

helm create makes a ready-made chart for a web server. The two sed lines change it to 2 copies of NGINX 1.27. (You can open loan-calc-chart/values.yaml in the Cloud Shell Editor to see the change.)

Verify the two values (Cloud Shell, home folder)

```
grep -E '^replicaCount|^  tag' loan-calc-chart/values.yaml
```

You should see:

```
replicaCount: 2
  tag: "1.27"
```

If either line is different, run the matching sed line again.

Step 2. Install (revision 1)

▸ Run in: Cloud Shell (Google Cloud console, the >_ icon at the top right), in the folder that contains loan-calc-chart (your home folder).

```
helm install loan-calc ./loan-calc-chart
helm list
```

You should see

```
NAME      NAMESPACE  REVISION  STATUS    CHART                APP VERSION
loan-calc default    1         deployed  loan-calc-chart-0.1.0  1.16.0
```

APP VERSION 1.16.0 is only the chart's label; the pods really run nginx 1.27 because of the tag you set.

Step 3. Upgrade (revision 2), then roll back (revision 3)

▸ Run in: Cloud Shell (Google Cloud console, the >_ icon at the top right), in the folder that contains loan-calc-chart (your home folder).

```
sed -i.bak 's/^replicaCount: .*/replicaCount: 3/' loan-calc-chart/values.yaml
helm upgrade loan-calc ./loan-calc-chart
helm rollback loan-calc 1
helm history loan-calc
```

You should see

```
REVISION  STATUS      CHART                DESCRIPTION
1         superseded  loan-calc-chart-0.1.0  Install complete
2         superseded  loan-calc-chart-0.1.0  Upgrade complete
3         deployed    loan-calc-chart-0.1.0  Rollback to 1
```

Only one revision is ever **deployed**. Take **Screenshot 2:** helm list and helm history loan-calc.

Verify the app after the rollback (Cloud Shell)

```
kubectl get deployment loan-calc-loan-calc-chart
kubectl get deployment loan-calc-loan-calc-chart -o jsonpath='{.spec.template.spec.containers[0].image}'
```

You should see READY **2/2** (back to revision 1's two copies, not 3) and the image **nginx:1.27**:

```
NAME                     READY   UP-TO-DATE   AVAILABLE
loan-calc-loan-calc-chart   2/2     2            2
nginx:1.27
```

Checkpoint: which revision is live?

```
helm history loan-calc | grep deployed
```

Revision 3, which is a copy of revision 1. A rollback never erases history; it adds a new revision. That history is what an auditor reads.

If not

Error: INSTALLATION FAILED: cannot re-use a name that is still in use: you installed before. Run `helm uninstall loan-calc`, then Step 2 again.

sed: can't read loan-calc-chart/values.yaml: you are in a different folder from where you ran `helm create`. Run `ls`; if `loan-calc-chart` is missing, run Step 1 again.

Part D. Lock it down with a NetworkPolicy (Security engineer, 15 min)

Goal: only pods labelled role=frontend may connect to the Helm release's pods, and two tests prove it: one blocked, one allowed.

Step 1. Get the policy and apply it

▸ Run in: Cloud Shell (Google Cloud console, the >_ icon at the top right), in your home folder (where Cloud Shell opens; type cd ~ first if you moved), the same folder as nginx-deployment.yaml.

```
cd ~
R=https://raw.githubusercontent.com/isijackson/IT360-Fall-Labs-v2/main
R="$R/Week%203%20labs/gke-helm-networkpolicy/k8s"
curl -fsSO "$R/network-policy.yaml"
cat network-policy.yaml
kubectl apply -f network-policy.yaml
kubectl get networkpolicy
```

Verify the download (Cloud Shell, home folder)

ls -l network-policy.yaml
You should see a size of **292** bytes. Then check the policy Kubernetes stored:

```
kubectl describe networkpolicy loan-calc-frontend-only
  PodSelector:     app.kubernetes.io/name=loan-calc-chart
  Allowing ingress traffic:
    From:
      PodSelector: role=frontend
  Policy Types: Ingress
```

Inside the file:

Inside the file (for reading only; do not paste this into Cloud Shell)

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: loan-calc-frontend-only
spec:
  podSelector:
    matchLabels:
      app.kubernetes.io/name: loan-calc-chart
  policyTypes:
  - Ingress
  ingress:
  - from:
    - podSelector:
        matchLabels:
          role: frontend
```

Read it as a sentence: "For the pods of the loan-calc chart, allow incoming traffic only from pods labelled role=frontend." Everything else is dropped.

You should see

```
networkpolicy.networking.k8s.io/loan-calc-frontend-only created
NAME                    POD-SELECTOR                          AGE
loan-calc-frontend-only   app.kubernetes.io/name=loan-calc-chart   0s
```

Step 2. Save the service name in a variable

▸ Run in: Cloud Shell (Google Cloud console, the >_ icon at the top right). Any folder, but keep using the same Cloud Shell tab for Steps 3 and 4 so $SVC is remembered.

```
SVC=$(kubectl get svc -l app.kubernetes.io/instance=loan-calc -o jsonpath='{.items[0].metadata.name}')
echo $SVC
```

You should see

loan-calc-loan-calc-chart

Step 3. Test 1: a pod WITHOUT the label must be blocked

▸ **Run in: Cloud Shell (Google Cloud console, the >_ icon at the top right), the same tab where you set $SVC.**

```
kubectl run probe --rm -it --image=busybox:1.36 --restart=Never -- wget -qO- --timeout=5 http://$SVC
```

You should see (this is a PASS)

wget: download timed out
pod "probe" deleted from default namespace
pod default/probe terminated (Error)

Timed out = blocked = what the auditor wants. The word Error only means wget gave up, which is the point. The command takes up to a minute because Autopilot may start a machine for the test pod first.

Step 4. Test 2: a pod WITH role=frontend must be allowed

▸ **Run in: Cloud Shell (Google Cloud console, the >_ icon at the top right), the same tab where you set $SVC.**

```
kubectl run probe-ok --rm -it --image=busybox:1.36 --restart=Never --labels=role=frontend -- wget -qO- --timeout=5 http://$SVC
```

You should see

<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
...

Take **Screenshot 3**: Test 1 and Test 2 in the same terminal window, one above the other.

Lines like `All commands and output from this session will be recorded in container logs` and `If you don't see a command prompt, try pressing enter.` are printed by GKE for every test pod. They are normal; just wait.

If not

Both tests print the NGINX page: the policy is not enforced. You probably created a Standard cluster. Autopilot enforces it; recreate the cluster as Autopilot (Part A, Option 1, step 3).

Both tests time out: check `echo $SVC` prints `loan-calc-loan-calc-chart`, and that the `--labels=role=frontend` part is in Test 2.

pods "probe" already exists: an earlier test is still there. `kubectl delete pod probe` and run the test again.

Part E. Test connectivity end to end (10 min)

Goal: check every path traffic can take, so you know not only that the app runs, but who can reach it. Compare your results with the instructor's in the last column.

▸ **Run every command in this table in Cloud Shell**, in the same tab where you set $SVC in Part D.

Path	Command	Instructor result (Nov 8)
Kubernetes heals a pod	kubectl delete pod <name> then kubectl get pods	New pod Running in 8 s
Pod finds the service by name (DNS)	kubectl run dns --rm -it --image=busybox:1.36 --restart=Never -- nslookup $SVC	Name resolves to the service IP
Pod without the label to the app	Test 1 in Part D	download timed out (blocked)
Pod with role=frontend to the app	Test 2 in Part D	Welcome to nginx! (allowed)
Pod to the unprotected kubectl app	kubectl expose deployment loan-calc --name=loan-calc-internal --port=80, then Test 1 with http://loan-calc-internal	Welcome to nginx!: the policy only guards the Helm pods

Optional (Canvas): open the app from the internet

▸ Run in: Cloud Shell (Google Cloud console, the >_ icon at the top right). Any folder. Then open the EXTERNAL-IP address in your laptop's web browser.

```
kubectl expose deployment loan-calc --type=LoadBalancer --name=loan-calc-public --port=80
kubectl get service loan-calc-public --watch
```

Wait until EXTERNAL-IP changes from <pending> to an address (44 seconds on the instructor run), press Ctrl+C, then open http://<EXTERNAL-IP> in your browser. The page may refuse to load for another minute or two while Google programs the load balancer; refresh before you change anything.

You should see

```
NAME              TYPE           CLUSTER-IP      EXTERNAL-IP    PORT(S)
loan-calc-public  LoadBalancer   34.118.232.243  34.68.49.240   80:31 278/TCP
```

In the browser: the **Welcome to nginx!** page.

Discuss as a team

Why does the public page load when the policy blocks pods? Because the LoadBalancer sends traffic to the loan-calc pods from Part B, and the policy only protects the Helm pods from Part C. Compare kubectl get svc loan-calc-public -o jsonpath='{.spec.selector}' with the policy's podSelector.

Stuck? Ask an AI assistant the right way

Open Gemini Cloud Assist (the spark icon at the top of the console). Paste the exact error text and the command you ran, and ask what caused it. **Never** paste keys, tokens or kubeconfig files. Check any command it suggests against this guide before you run it.

Part F. Submit, then clean up (10 min)

What to submit in Canvas 3.2

- ☐ **Screenshot 1:** gcloud container clusters list showing RUNNING, and kubectl get pods with three pods Running after you deleted one.
- ☐ **Screenshot 2:** helm list and helm history loan-calc (revision 3, Rollback to 1).
- ☐ **Screenshot 3:** Test 1 (timed out) and Test 2 (Welcome to nginx!) in one terminal.
- ☐ The NetworkPolicy YAML (copy it from Part D).
- ☐ A 2 to 3 sentence summary that names who did what (one sentence per role is fine).

Clean up the same day

An Autopilot cluster costs about 73 dollars a month even when idle. Run these in order once your screenshots are saved:

▸ Run in: Cloud Shell (Google Cloud console, the >_ icon at the top right), in your home folder (where Cloud Shell opens; type cd ~ first if you moved), where the two YAML files are.

```
kubectl delete service loan-calc-public loan-calc-internal --ignore-not-found
helm uninstall loan-calc
kubectl delete -f network-policy.yaml -f nginx-deployment.yaml
gcloud container clusters delete silverlake-public --region us-central1 --quiet
gcloud container clusters list
```

You should see

The last command prints nothing: no clusters left. Then end the Vocareum lab in Canvas.

Delete the LoadBalancer service **before** the cluster, so Google also removes the load balancer it created for you.

Troubleshooting at a glance

You see	Do this
does not have enough resources available	Recreate the cluster with --region us-east1 and use that region everywhere.
Pods or test pods stay Pending	Autopilot is adding a machine. Wait up to 90 seconds; kubectl get events shows progress.
Both tests succeed	Not Autopilot, so the policy is not enforced. Recreate as Autopilot.
helm install says the name is in use	helm uninstall loan-calc, then install again.
kubectl prints HTML or times out	Run gcloud container clusters get-credentials silverlake-public --region us-central1 again.
Cloud Shell disconnected and $SVC is empty	Run Part D Step 2 again; variables do not survive a reconnect.
EXTERNAL-IP stays <pending> over 3 minutes	kubectl describe service loan-calc-public and read the Events at the bottom.

Words to know

Word	Meaning in this lab
Cluster	The set of machines Kubernetes runs your app on. Autopilot manages the machines for you.
Pod	One running copy of the app (here: one NGINX web server).
Deployment	The instruction "keep N pods of this running". It replaces pods that die.
Service	A stable name and address in front of a group of pods.
Helm release	An installed app with a numbered history, so you can roll back.
NetworkPolicy	A firewall rule between pods, chosen by labels instead of IP addresses.
Label	A tag on a pod, such as `role=frontend`. Policies and Services select pods by label.